

Reviewer Pack — Minimal Order of Quadratic Residues and SAT Clause Set Entro...

Entry b984679c3611 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 00:44:22 UTC

Minimal Order of Quadratic Residues and SAT Clause Set Entropy Correlation

Entry ID: b984679c3611 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 00:44:22 UTC

1. Conjecture

Field A (mathematical branch): Quadratic Number Theory **Field B** (complexity object): SAT Clause Sets

Statement:

For every CNF formula φ with n variables, the minimal order of a quadratic residue modulo a prime p used to construct an isomorphism between φ and its dual φ^* ($\text{min_order}(\varphi)$) is linearly correlated with the entropy of φ 's clause set $H(\varphi)$, such that $\text{min_order}(\varphi) = \Theta(H(\varphi))$.

Rationale (proposer's reasoning):

Quadratic number theory offers a rich source of invariant quantities related to modular arithmetic, which could potentially reveal structural properties in Boolean functions. The correlation between these mathematical invariants and the entropy of SAT clause sets could expose novel relationships between algebraic structure and computational complexity.

Taxonomy category: Quadratic Number Theory $\times$ SAT Clause Sets (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 90f263cf8007e220

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF formula φ with n variables, if the Pearson correlation coefficient (r) between $\text{min_order}(\varphi)$ and $H(\varphi)$ is $r \geq 0.7$ for at least 25 out of 30 random seeds, it supports the conjecture; if any seed produces an absolute value of $r < 0.5$ or an outlier with $|\text{min_order}(\varphi) - \Theta(H(\varphi))| > 2$ standard deviations from the mean, it falsifies the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def next_prime(p):
        p += 1
        while not is_prime(p):
            p += 1
        return p

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n-1, i-1, -1):
                A[i][j] /= A[i][i]
```

```

        for k in range(m):
            if k != i and A[k][i] != 0:
                factor = A[k][i]
                for j in range(i, n):
                    A[k][j] -= factor * A[i][j]
    return A

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0]*p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def min_order(phi, p):
    phi = [int(x) for x in phi.split()]
    n = len(phi)
    for i in range(1, p):
        if pow(i, 2, p) == 1:
            return i
    return None

def shannon_entropy(phi):
    counts = {}
    for clause in phi:
        if clause not in counts:
            counts[clause] = 0
        counts[clause] += 1
    total = sum(counts.values())
    entropy = 0
    for count in counts.values():
        p = Fraction(count, total)
        entropy -= p * math.log2(p)
    return entropy

def generate_cnf(n):
    clauses = []
    for _ in range(2**n):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(random.randint(1, n))]
        if all(c != -x for c, x in zip(clause, clause)):
            clauses.append(tuple(sorted(clause)))
    return clauses

def is_isomorphic(phi, psi):
    phi = sorted(phi)
    psi = sorted(psi)
    if len(phi) != len(psi):
        return False
    mapping = {}
    for i in range(len(phi)):
        if phi[i] not in mapping and psi[i] not in mapping.values():
            mapping[phi[i]] = psi[i]
        elif phi[i] not in mapping or mapping[phi[i]] != psi[i]:
            return False

```

```

    return True

def dual_cnf(phi):
    dual = []
    for clause in phi:
        dual.append(tuple(-x for x in clause))
    return dual

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_min_order = 0
total_entropy = 0
min_order_values = []
entropy_values = []

for n in n_values:
    for _ in range(5):
        phi = generate_cnf(n)
        psi = dual_cnf(phi)
        p = next_prime(2*n)
        min_order_phi = min_order(phi, p)
        min_order_psi = min_order(psi, p)
        if min_order_phi is None or min_order_psi is None:
            continue
        instances_tested += 1
        total_min_order += (min_order_phi + min_order_psi) / 2
        entropy_phi = shannon_entropy(phi)
        entropy_psi = shannon_entropy(psi)
        total_entropy += (entropy_phi + entropy_psi) / 2
        min_order_values.append((min_order_phi + min_order_psi) / 2)
        entropy_values.append((entropy_phi + entropy_psi) / 2)

if instances_tested < 30:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not enough instances"
    }

mean_min_order = total_min_order / instances_tested
mean_entropy = total_entropy / instances_tested

n = len(min_order_values)
sum_diff_squared = sum((min_order_values[i] - mean_min_order) * (entropy_values[i] - mean_entropy) for i in range(n))
sum_diff_min_order_squared = sum((min_order_values[i] - mean_min_order) ** 2 for i in range(n))
sum_diff_entropy_squared = sum((entropy_values[i] - mean_entropy) ** 2 for i in range(n))

if sum_diff_min_order_squared == 0 or sum_diff_entropy_squared == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": max(n_values),

```

```

        "conjecture_holds": False,
        "counterexample": "zero_variance"
    }

r = sum_diff_squared / math.sqrt(sum_diff_min_order_squared * sum_diff_entropy_squared)

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": r,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": abs(r) >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(result["metric_value"] is not None for result in results):
        mean_r = sum(result["metric_value"] for result in results) / len(results)
        std_r = math.sqrt(sum((result["metric_value"] - mean_r) ** 2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if abs(result["metric_value"]) >= 0.7) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_r} std={std_r} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["metric_value"] is None)
        print(f"RESULT: INCONCLUSIVE reason=missing_data first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d4a1929.py", line 180, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d4a1929.py", line 123, in run_trial
    min_order_phi = min_order(phi, p)
                     ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3d4a1929.py", line 64, in min_order
    phi = [int(x) for x in phi.split()]
           ~~~~~
AttributeError: 'list' object has no attribute 'split'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the correlation between $\min_order(\varphi)$ and $H(\varphi)$. As a result, we cannot confirm or falsify the conjecture based on the provided information. | next: Investigate the cause of the crash in the test code and attempt to run the test again. If the issue can be resolved, re-evaluate the correlation between $\min_order(\varphi)$ and $H(\varphi)$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26352
2	preregistration	ollama_remote	glm4:latest	0	0	16376
3	novelty	ollama_remote	glm4:latest	0	0	8617
4	novelty	ollama_remote	glm4:latest	0	0	8474
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23277
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16794
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20443
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	86287
9	judge	ollama_remote	glm4:latest	0	0	63388

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 270008 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b984679c3611.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b984679c3611.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b984679c3611.tar.gz` (if generated)